



BIG DATA INTERVIEW SERIES

Hadoop MapReduce Interview Tutorial

From core concepts to the Word Count flow, architecture, fault tolerance, and the interview questions that come up most

Map & Reduce

Shuffle & Sort

YARN Architecture

MapReduce vs Spark

Prepared for interview practice · Hadoop ecosystem & PySpark track

1 What is MapReduce?

MapReduce is a distributed data processing programming model and execution framework used in Hadoop to process very large datasets across multiple machines. It divides a large job into two main phases.

```
Input Data
  ↓
  MAP
  ↓
Intermediate Key-Value Pairs
  ↓
Shuffle and Sort
  ↓
  REDUCE
  ↓
Final Output
```

CORE IDEA

Map: Transform input data into key-value pairs. **Reduce:** Aggregate values that share the same key.

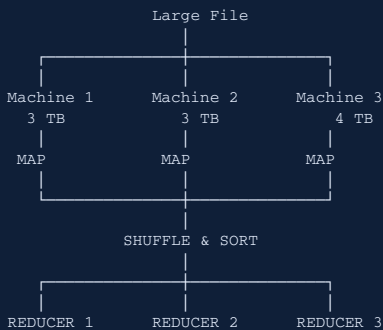
2 Why Do We Need MapReduce?

Suppose we have a file containing `hello hadoop / hello spark / hadoop spark` and we want to count every word. A traditional program processes the file on one machine — fine for small files, but imagine a file with **10 TB of data**. One machine would be far too slow.

Single Machine (Bad for Big Data)

```
Machine 1
  ↓
Read entire file
  ↓
Process entire file
  ↓
Return result
```

Hadoop's Approach: Parallel Processing



3 The MapReduce Data Flow

```
Input Data
  ↓
InputFormat
  ↓
Input Splits
  ↓
RecordReader
  ↓
Mapper
  ↓
Combiner (Optional)
  ↓
Partitioner
  ↓
Shuffle and Sort
  ↓
Reducer
  ↓
OutputFormat
  ↓
HDFS Output
```

MOST IMPORTANT STAGES FOR AN INTERVIEW

Mapper → Combiner → Partitioner → Shuffle & Sort → Reducer

4 The Word Count Example

Given three input files:

File 1: hello hadoop

File 2: hello spark

File 3: hadoop spark

We want the output: **hello 2, hadoop 2, spark 2**

5 Step 1: Input Data

Hadoop stores the data in HDFS, divided across blocks:

```
HDFS
├── Block 1
│   └── hello hadoop
├── Block 2
│   └── hello spark
└── Block 3
    └── hadoop spark
```

The data is then divided into **input splits**. Usually: **1 Input Split → 1 Mapper Task**

INTERVIEW POINT

An input split is a logical division of input data used by a mapper.

6 Step 2: Mapper

The Mapper receives data as <key, value>. For a text file: **Key** = line offset, **Value** = line content.

Key = 0 **Value** = "hello hadoop"

The mapper splits each line into words:

Key	Value
hello	1
hadoop	1
hello	1
spark	1
hadoop	1
spark	1

7 Step 3: Combiner

The Combiner is an **optional** local aggregation step. Instead of sending two separate <hello, 1> pairs over the network, it can calculate <hello, 2> locally — reducing network traffic.

Is the Combiner guaranteed to run?

No. Hadoop may execute it **0 times, 1 time, or multiple times**. Therefore, the Combiner logic must only be used when the operation is safe for repeated execution.

SAFE EXAMPLE

SUM

RISKY EXAMPLE

AVERAGE — averaging partial averages incorrectly produces wrong results.

8 Step 4: Partitioner

The Partitioner determines **which Reducer receives each key**. The default partitioner usually uses:

`hash(key) % number_of_reducers`

Key	Destination
hello	Reducer 1
hadoop	Reducer 0
spark	Reducer 2

RULE
The same key must always go to the same Reducer, since the Reducer needs all values for that key together (e.g. hello → [1, 1]).

9 Step 5: Shuffle and Sort

One of the most important phases in MapReduce — the data is grouped by key.

Key	Values
hadoop	[1, 1]
hello	[1, 1]
spark	[1, 1]

The Reducer does **not** receive individual records like two separate <hello, 1> pairs — instead it receives hello → [1, 1].

10 Step 6: Reducer

The Reducer aggregates the grouped values using sum(values):

Key	Output
hadoop	2
hello	2
spark	2

11 Complete Word Count Flow



12 MapReduce Components

1. InputFormat

Determines how the input data is split and how records are read. Default: **TextInputFormat**, producing <line offset, line text> e.g. <0, "hello hadoop">.

2. InputSplit

A logical portion of the input data — e.g. a 1 GB file split into three 128 MB splits.

INTERVIEW POINT

The number of Mappers is usually related to the number of InputSplits, not directly to the number of HDFS blocks.

3. RecordReader

Converts the input split into key-value pairs, e.g. Key = 0, Value = "hello hadoop".

4. Mapper

Transforms input records <0, "hello hadoop"> into <hello, 1>, <hadoop, 1>.

5. Combiner (Optional)

Performs local aggregation: two <hello, 1> become one <hello, 2>.

6. Partitioner

Decides which Reducer receives each key via $\text{hash}(\text{key}) \% \text{number_of_reducers}$.

7. Shuffle and Sort

Moves Mapper output, groups identical keys, and sorts them — e.g. two hello → 1 become hello → [1, 1].

8. Reducer

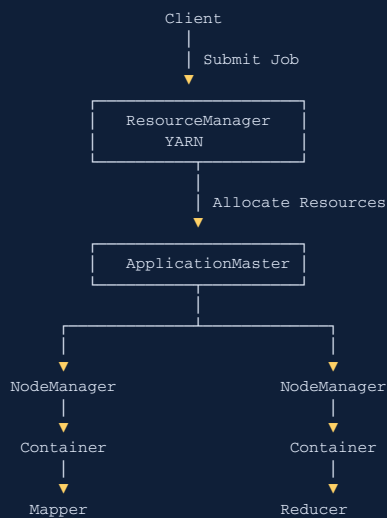
Processes grouped data: hello → [1, 1] becomes hello → 2.

9. OutputFormat

Writes the final output to HDFS. Default: **TextOutputFormat**, format key<TAB>value.

13 MapReduce Architecture

In modern Hadoop, MapReduce runs on top of **HDFS + YARN + MapReduce**.



14 How a MapReduce Job Runs

Step 1 — Client submits the job

The client sends the MapReduce Job to the ResourceManager.

Step 2 — ResourceManager starts ApplicationMaster

The ApplicationMaster manages the specific job and requests resources from YARN.

Step 3 — Input data is divided

Hadoop creates Input Splits, e.g. Split 1 → Mapper 1, Split 2 → Mapper 2, Split 3 → Mapper 3.

Step 4 — YARN allocates containers

Containers are provided for Mapper Tasks and Reducer Tasks.

Step 5 — Mappers process data

Input Split → Mapper → Intermediate Key-Value Pairs.

Step 6 — Shuffle and Sort

Mapper output is transferred to the appropriate Reducer (Mapper 1, 2, 3 → Shuffle → Reducer).

Step 7 — Reducers produce final output

Reducer → HDFS, e.g. `/user/youssef/output/part-r-00000`.

15 Map Task vs Reduce Task

Map Task	Reduce Task
Processes input data	Processes intermediate data
Runs close to input data	Receives data from Mappers
Produces key-value pairs	Aggregates values
Usually more numerous	Usually fewer
First major processing phase	Final major processing phase

```
Map:                                Reduce:

"hello world"                       hello → [1, 1, 1]
  ↓                                  ↓
hello → 1                           hello → 3
world → 1
```

16 What Happens If a Node Fails?

Hadoop MapReduce provides fault tolerance. Suppose **Mapper 1** fails — the ApplicationMaster detects the failure, then Hadoop:

1. Detects the failed task
2. Starts the task again
3. Runs it on another available node

Because input data is stored in HDFS with replication:

```
Block Replica 1 → DataNode 1
Block Replica 2 → DataNode 2
Block Replica 3 → DataNode 3
```

INTERVIEW POINT

Hadoop achieves fault tolerance through HDFS data replication and MapReduce task re-execution.

17 Data Locality

Hadoop tries to move **Computation → Data** instead of **Data → Computation**.

Bad Approach

```
DataNode 1
  |
  | Transfer 1 TB
  ↓
Processing Node (very expensive)
```

Hadoop Approach

```
DataNode 1
  |
  | Run Mapper locally
```

1. Node Local — best

2. Rack Local

3. Off Rack

18 Important MapReduce Interview Questions

Q1. What is MapReduce?

A distributed processing framework and programming model in Hadoop used to process large datasets in parallel across multiple nodes. It consists mainly of a Map phase (transforms input into intermediate key-value pairs), Shuffle and Sort (groups data by key), and a Reduce phase (aggregates the grouped values).

Q2. What is the difference between Mapper and Reducer?

The Mapper processes input data and generates intermediate key-value pairs. The Reducer receives all values for the same key after shuffle and sort, and performs aggregation or final processing.

Q3. What is Shuffle and Sort?

The phase that transfers Mapper output to the appropriate Reducers, groups records with the same key, and sorts the keys before the Reducer processes them.

Q4. What is a Combiner?

An optional local aggregation function that runs after the Mapper and before the Shuffle phase, reducing data transferred across the network.

Q5. Is the Combiner guaranteed to execute?

No — Hadoop may execute it zero, one, or multiple times, so its logic must be safe for repeated execution.

Q6. How many Mappers are created?

Mainly determined by the number of InputSplits. Usually 1 InputSplit $\approx$ 1 Mapper.

Q7. How many Reducers are created?

Configured by `mapreduce.job.reduces`, e.g. `-D mapreduce.job.reduces=3`. If set to 0, the job becomes a Map-only job.

Q8. What happens if there are no Reducers?

The flow becomes Input $\rightarrow$ Mapper $\rightarrow$ Output directly (e.g. CSV $\rightarrow$ Mapper $\rightarrow$ Cleaned CSV). This is a Map-only job with no Shuffle, Sort, or Reduce phase.

Q9. What is the default number of Reducers?

Default is 1, though in real projects the number is configured according to the workload.

Q10. Why is MapReduce slow compared to Spark?

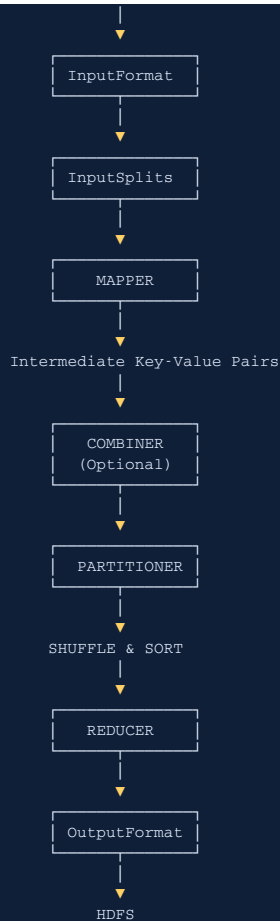
MapReduce writes intermediate data to disk between stages (Map $\rightarrow$ Disk $\rightarrow$ Shuffle $\rightarrow$ Disk $\rightarrow$ Reduce), while Spark keeps intermediate data in memory (Map $\rightarrow$ Memory $\rightarrow$ Transformation $\rightarrow$ Reduce). MapReduce is disk-based; Spark can also use disk when necessary but is primarily memory-based.

19 MapReduce vs Spark

Feature	MapReduce	Spark
Processing	Disk-based	In-memory primarily
Speed	Slower	Faster
Programming	More complex	Easier
Iterative algorithms	Poor	Excellent
Machine Learning	Less suitable	Better
Streaming	Limited	Better
Fault tolerance	Replication + re-execution	Lineage + re-computation

MapReduce is disk-oriented and processes data in distinct Map and Reduce stages. Spark uses in-memory computation and supports more flexible processing models such as batch processing, streaming, SQL, machine learning, and graph processing.

20 The Most Important Diagram to Memorize



🔧 Your Interview Study Order

Level 1 — Understand the concept

What is MapReduce? · Why do we need it? · Map vs Reduce

Level 2 — Understand the flow

Input → InputSplit → Mapper → Combiner → Partitioner → Shuffle & Sort → Reducer → Output

Level 3 — Understand Hadoop integration

HDFS → YARN → MapReduce

Level 4 — Practice, build these projects:

Word Count

Average Temperature

Maximum Temperature

Sales Aggregation

Customer Count by Country

Log Analysis

Top 10 Products by Sales

Next step: build a complete MapReduce project using a real CSV dataset, then compare the same project with PySpark.